INF1103 Phase 2 / Engineering report

DoNotStress • Team 1 • Object-Oriented Refactoring and Extension • DRAFT for team verification

Purpose. Refactor the existing AI-assisted student wellbeing check-in into a coherent object-oriented application while preserving the original Gemini-dependent scoring pipeline. The application is non-diagnostic and uses a protected admin history interface.

1. System architecture

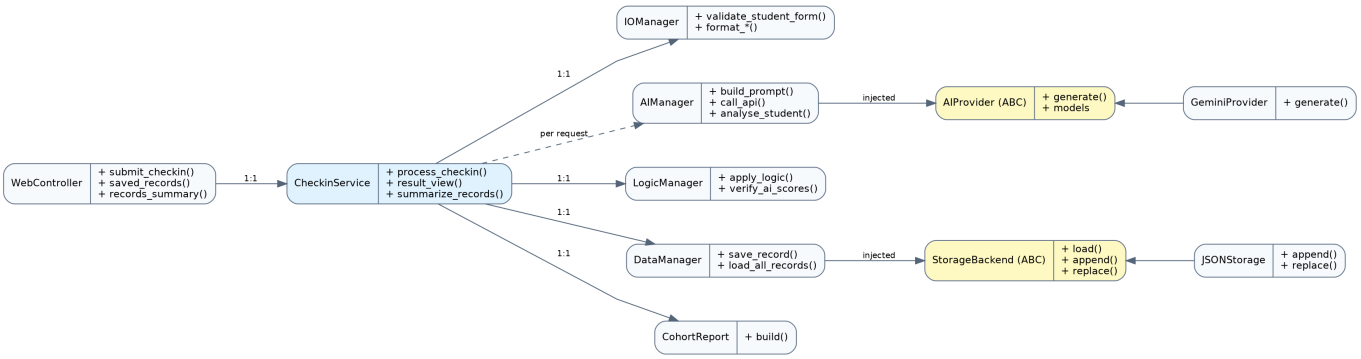


Figure 1. OOP architecture preview. The complete class/member-level UML is supplied in docs/UML_ALL_CLASSES.puml.

Responsibility	Implementation / interface
IO Manager	IOManager validates IDs and 11 answers; formats student-facing results and CLI output.
AI Manager	AIManager uses injected AIProvider; GeminiProvider sends structured JSON; validation and retry remain mandatory.
Logic Manager	LogicManager audits AI arithmetic with StressScorer and applies polymorphic BusinessRule objects.
Data Manager	DataManager enforces AI/opt-in rules, filters private text, and uses StorageBackend/JSONStorage.

Application boundary. main.create_app builds the managers, CheckinService and WebController. The Flask routes invoke object methods. Old module entry points remain solely as compatibility adapters for original tests.

2. Data flow and failure boundaries

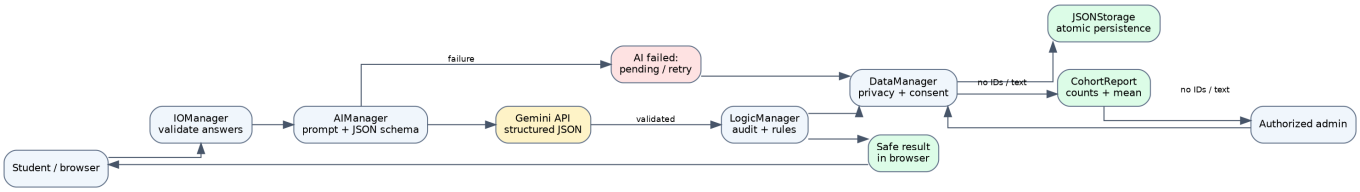


Figure 2. Mandatory AI-first processing, safe pending handling, persistent records and admin summaries.

Record lifecycle

1. Validate. WebController accepts HTTP input, and IOManager validates a seven-digit Student ID, eleven required answers and optional reflection. The reflection is checked locally for basic safety phrasing but never included in the Gemini prompt.

2. Enrich and audit. CheckinService passes validated answers to AIManager. GeminiProvider requests structured JSON; AIManager validates it. LogicManager checks that the AI score is consistent with the deterministic formula and runs the domain rules. AI failures never become evaluated results.

3. Store or retry. Evaluated records are stored by DataManager only after domain validation. When AI fails, the application can store a clearly marked pending record and invite a retry. JSONStorage uses temporary files and atomic replace; malformed files are backed up. Unexpected JSON schemas are rejected to prevent loss.

4. Present. IOManager formats an allow-listed student result. Authenticated admins can query saved numeric responses and request a cohort summary; no private reflection or raw AI reasoning appears in the report endpoint.

Domain scoring invariants

Rule	Invariant
11-answer stress scale	Mean of 11 converted values, 1-5; positive wording reverse-scored. Sleep hours map into five bands.
AI cross-check	Gemini must supply required fields. Score mismatch beyond 0.01 is refused; no fabricated fallback score.
Context routing	Safety override, support priority, context flags and tip cap are separate ordered BusinessRule objects.
Cohort summary	Only valid evaluated scores enter mean stress; pending and invalid scores cannot distort the average.

3. UML and mandatory OOP concepts

The full class/member UML is in docs/UML_ALL_CLASSES.puml; the readable relationship/multiplicity diagram is docs/UML_CORE.puml. The following is the mapping to actual implementation, including private attributes.

Concept	Code-based evidence
Classes + objects	main.create_app constructs IOManager, LogicManager, DataManager, CheckinService and WebController; each request receives an AIManager.
Encapsulation	AIManager.__provider, DataManager.__backend, CheckinService.__logic and IOManager.__input are private; public methods provide access.
Abstract class	BaseManager(ABC).describe; AIProvider, StorageBackend, Scorer and BusinessRule use abstract methods.
Inheritance	Four manager classes extend BaseManager; GeminiProvider extends AIProvider; StressScorer extends Scorer.
Interface / contract	AIProvider.generate, StorageBackend.load/append/replace, BusinessRule.run; referenced through abstract types.
Association	CheckinService collaborates with injected IOManager, LogicManager and DataManager that can exist separately.
Aggregation	AIManager accepts an independent AIProvider; LogicManager holds externally supplied BusinessRule instances.
Composition	LogicManager creates default StressScorer and each Decision; CheckinService creates default CohortReport.
Polymorphism	AIManager accepts GeminiProvider or FunctionProvider; DataManager accepts JSONStorage or test memory storage; rule.run handles different subclasses.

SOLID (required: SRP, OCP and LSP)

Single Responsibility. IO validates/presents, AI owns prompt/API/retry/schema, Logic checks and routes, Data persists/queries, WebController handles HTTP/session concerns, and CohortReport computes only aggregates.

Open/Closed. A new provider implements AIProvider; a new store implements StorageBackend; a new rule subclasses BusinessRule. No manager-specific type branching is required for those extensions.

Liskov Substitution. Offline fake and live providers implement the same AIProvider contract; JSON and memory backends implement the same StorageBackend contract. Tests substitute these instances without changing client code.

4. Features, tests and DevOps

Meaningful new feature: cohort overview

Admin users can view total, evaluated and pending check-ins, Low/Moderate/High tier distribution, mean stress for eligible evaluated records and a per-cohort breakdown. These aggregates are produced by CohortReport.build, invoked through CheckinService, served by WebController at GET /api/summary, and displayed in static/campus.js. Student IDs, reflections and AI reasoning are omitted.

Advanced feature: graphical user interface

Flask templates and JavaScript provide a multi-step student survey, a student-safe result, administrator sign-in and protected saved-record filters. Team verification: confirm whether GUI qualifies as a new advanced feature relative to Phase 1; obtain instructor agreement if necessary.

Testing, environment and repeatability

Verification item	Evidence / status
Python syntax	Passed: python -m compileall -q .
JavaScript syntax	Passed: node --check static/campus.js
Selected offline unit tests	76 / 76 passed; includes existing manager tests and new OOP, privacy and data-integrity regressions.
HTTP tests with Flask	Added/retained; not executed here because Flask was unavailable.
Real Gemini / SDK	Preserved integration; live request not verified here.
Docker / CI	Dockerfile and .github/workflows/tests.yml included. Successful Docker and hosted CI runs not verified.

Run on a configured submission machine

```
python -m pip install -r requirements.txt
python -m unittest discover -s tests -v
docker build -t donotstress-phase2 .
docker run --rm donotstress-phase2 python -m unittest discover -s tests -v
```

All mocked tests must be independent of a live AI service. Never commit credentials or genuine student records; keep the file path outside version control. Real Gemini must still be exercised separately with a secured key.

5. Exception-handling matrix

Failure	Response and evidence
Missing Gemini key	AIManager returns missing_api_key, not a fabricated successful assessment.
Timeout, 429 or API error	AIManager classifies failures and retries/falls back within four total provider calls.
Malformed AI JSON/schema	Schema validator rejects invalid fields; no evaluated result is persisted.
Logic audit mismatch	LogicManager rejects inconsistent scores rather than replacing AI output.
Invalid form / large request	IOManager returns field errors (HTTP 400); Flask handles HTTP 413.
Missing/corrupt data	Missing = empty; corrupt = backup and error on read. Write errors return a retryable failure.
Unsupported JSON / mixed list	Backup original and refuse overwrite, protecting unknown records.
Unauthorized admin / invalid CSRF	Session check before reading data; 401/redirect. Invalid POST token -> 403.

Final submission evidence checklist

Not supplied and not invented: actual repository URL/commit SHA, chronological contributor commit history, exported TeamAId task CSV, peer evaluation, a successful hosted CI run, and a full Docker integration run. The team must add genuine evidence before final submission.

Submission: Engineering report (maximum six pages); runnable OOP code; automated tests; root Dockerfile; Git repository/history; TeamAId CSV as a separate file. Final submission: Wednesday 18 November 2026, 23:59 Singapore time. Package using the official lab-group/team ZIP naming convention.

Risk boundary: The project remains a local coursework prototype. Fixed demonstration admin credentials, automatic save semantics and single-process session state must be reviewed before handling real student wellbeing records in a production deployment.

Report status: Working draft derived from the refactored implementation. Review with your team, insert actual test/Git/TeamAId evidence, and confirm that all statements match the submitted branch.